

7.1 tidyr: Tidy Data and pivot_longer()

One variable per column, one observation per row. pivot_longer() turns wide week columns into rows.

1. Tidy data

A table is **tidy** when:

1. each variable is a column
2. each observation is a row
3. each value is a cell

That is a shape, not a quality judgment. A tidy table can still have missing values, typos, and bad types. A **clean** table has those problems handled. You usually tidy first, then clean.

tidyr is the tidyverse package for the shape. It loads with library(tidyverse).

```
library(tidyverse)
```

2. table1 is tidy; table4a is not

tidyr ships small WHO tuberculosis tables. ?table1.

```
table1  
table4a
```

table1 has country, year, cases, and population. One year is one row. You can compute a rate without extra reshaping:

```
table1 |>  
  mutate(rate = cases / population * 10000)
```

table4a stores **years as column names** (1999, 2000) and cases as the cells. Year is a variable, so those names are values in disguise. You cannot filter(year == 1999) until year is a column.

table4b is the same wide layout for population.

3. pivot_longer() makes columns into rows

Three arguments you will type every time:

- `cols` — which columns are values of one variable
- `names_to` — the new column that holds the old names
- `values_to` — the new column that holds the cells

```
table4a |>
  pivot_longer(
    cols = c(`1999`, `2000`),
    names_to = "year",
    values_to = "cases"
  )
```

The result has one row per country-year. `year` is character because it came from names. `mutate(year = as.integer(year))` if you need a number.

`cols` uses the same helpers as `select()`. `cols = -country` is the usual shortcut when every other column should lengthen.

```
table4a |>
  pivot_longer(cols = -country, names_to = "year", values_to = "cases")
```

4. Numeric-looking names need backticks

`1999` is a number. A bare number in `cols` is a **position**, not a name. Column 1999 does not exist.

This also fails for a second reason: without `c()`, `2000` is a separate argument, not part of `cols`.

```
table4a |>
  pivot_longer(cols = 1999, 2000, names_to = "year", values_to = "cases")
```

Write ``1999`` so R treats it as a name, and wrap the names in `c()`. The same backtick rule applies in `select()` and `mutate()`.

`values_drop_na = TRUE` drops rows whose cell was `NA`. Use it when a missing cell means "this combination was never measured," not "the value is unknown."

```
table4a |>
  pivot_longer(
    cols = -country,
    names_to = "year",
    values_to = "cases",
    values_drop_na = TRUE
  )
```

5. Practice

Put course CSV files in a `data/` folder of your project (see the Data section of the course page).

1. Load `monkeymem.csv`. The `Week2 ... Week16` columns are weeks; the cells are accuracy. Pivot them longer so each row is one monkey-week.
2. On `table4a`, explain why `pivot_longer(cols = 1999, 2000, names_to = "year", values_to = "cases")` fails. Write the version that works.